

Numerical Methods — Questions & Answers

Based on the Numerical Methods course content (Trapezoidal, Simpson's, root-finding, interpolation, and related topics).

1. What is Numerical Analysis? Applications & Limitations

Numerical Analysis is the branch of mathematics that develops and studies algorithms for obtaining approximate numerical solutions to mathematical problems that are difficult or impossible to solve exactly (analytically) — such as non-linear equations, systems of equations, integrals, derivatives, and differential equations.

Applications:

- Engineering simulations (structural, fluid, thermal analysis)
- Solving non-linear equations (root finding) in physics/engineering design
- Interpolation/curve fitting for experimental data
- Numerical integration/differentiation when a function has no closed-form antiderivative
- Computer graphics, weather prediction, financial modeling

Limitations:

- Results are approximate, not exact — always carry some error (truncation/round-off)
- Convergence isn't guaranteed for every method/every starting point
- Sensitive to step size, initial guess, and precision of arithmetic (round-off accumulation)
- Computationally expensive for large systems compared to direct/analytical methods when those exist

2. All Root-Finding Methods — Algorithms

From the course material:

Bisection Method

- Define $f(x)$; find a, b with $f(a) < 0, f(b) > 0$
- $c = (a + b) / 2$
- If $f(c) < 0$, set $a = c$; else set $b = c$
- Repeat until desired accuracy

False Position (Regula Falsi) Method

- Define $f(x)$; find a, b with $f(a) < 0, f(b) > 0$
- $c = [a \cdot f(b) - b \cdot f(a)] / [f(b) - f(a)]$
- If $f(c) < 0$, set $a = c$; else set $b = c$
- Repeat until desired accuracy

Newton-Raphson Method

- Define $f(x)$ and $f'(x)$

- Choose x_0 (often midpoint of bracketing interval)
- $x_{n+1} = x_n - f(x_n) / f'(x_n)$
- Repeat until $|x_{n+1} - x_n| \leq \text{tolerance}$

Fixed Point Iteration Method

- Rewrite $f(x) = 0$ as $x = g(x)$
- Choose x_0 such that $|g'(x_0)| < 1$
- $x_{n+1} = g(x_n)$
- Repeat until convergence

3. When Does the Answer Converge in a Root-Finding Method?

Convergence means successive approximations get progressively closer to the true root — measured by $|x_{n+1} - x_n|$ (or $|f(x_n)|$) becoming smaller than a set tolerance, as seen throughout the worked examples (e.g., stopping when $|x_{13} - x_{12}| < 10^{-6}$).

Conditions:

- **Bisection/False Position:** Converge as long as the initial interval $[a, b]$ brackets a root with $f(a)$ and $f(b)$ having opposite signs, and f is continuous — guaranteed convergence (though sometimes slow).
- **Newton-Raphson:** Converges (quadratically — very fast) if the initial guess is sufficiently close to the root and $f'(x) \neq 0$ near the root.
- **Fixed Point Iteration:** Converges only if $|g'(x)| < 1$ in the neighborhood of the root (this is why the worked solutions always check $|g'(x_0)| < 1$ before iterating).

4. Absolute Error vs Relative Error

- **Absolute Error** = $|\text{true value} - \text{approximate value}|$. It has the same units as the quantity itself and doesn't reflect how "significant" the error is relative to the size of the number.
- **Relative Error** = $\text{Absolute Error} / |\text{true value}|$ (often expressed as a percentage). It normalizes the error, making it meaningful for comparing accuracy across different magnitudes.

Example: an error of 1 is huge if the true value is 2, but negligible if the true value is 1,000,000 — relative error captures that difference while absolute error does not.

5. What is a Fixed Point / Fixed Point Method? When Does It Diverge?

A **fixed point** of a function $g(x)$ is a value x^* such that $x^* = g(x^*)$ — i.e., the input equals the output.

The **Fixed Point Iteration Method** solves $f(x) = 0$ by rewriting it as $x = g(x)$, then iterating $x_{n+1} = g(x_n)$ starting from a guess x_0 (e.g., $x^2 - x - 1 = 0$ rewritten as $x = \sqrt{x + 1}$).

Divergence occurs when $|g'(x)| \geq 1$ near the root — the iterates move progressively further from the true root instead of closer. This is why the algorithm explicitly requires checking $|g'(x_0)| < 1$ before proceeding; the choice of how you rearrange $f(x) = 0$ into $x = g(x)$ matters enormously, since different rearrangements of the same equation can have wildly different $g'(x)$ values.

6. When Does the Answer Not Converge (Diverge)?

General causes of divergence across root-finding methods:

- **Fixed Point Iteration:** $|g'(x)| \geq 1$ near the root (as above)

Newton-Raphson:

- $f'(x)$ is zero or very close to zero near the guess (causes huge or undefined jumps)
- Initial guess too far from the true root
- The function has an inflection point or multiple roots close together, causing oscillation
- **Bisection/False Position:** These generally won't diverge if the initial bracket is valid ($f(a)$ and $f(b)$ opposite signs) and f is continuous — but False Position can converge very slowly if the function is heavily skewed (one endpoint barely moves for many iterations).
- General numerical issue: accumulated round-off error can prevent true convergence to machine precision even when the method is theoretically convergent.

7. Numerical Differentiation & Integration — Definitions

- **Numerical Differentiation** approximates the derivative of a function using discrete data points (rather than symbolic calculus), typically via finite-difference formulas, e.g., the central difference: $f'(x) \approx [f(x+h) - f(x-h)] / 2h$, as used in the Richardson extrapolation example.
- **Numerical Integration (Quadrature)** approximates the definite integral of a function by evaluating it at a finite set of points and combining those values using weighted sums — used when a function can't be integrated analytically or is only known at discrete data points. Examples: Trapezoidal rule, Simpson's rules.

8. Trapezoidal, Simpson's 1/3, Simpson's 3/8 Rules — Definitions

- **Trapezoidal Rule:** Approximates the area under a curve by dividing it into n subintervals and treating each as a trapezoid (straight line connecting consecutive points). Formula: $I = (h/2)[y_0 + y_n + 2(y_1 + y_2 + \dots + y_{n-1})]$.
- **Simpson's 1/3 Rule:** Approximates the curve between points using parabolic (quadratic) segments instead of straight lines, giving higher accuracy. Formula: $I = (h/3)[(y_0 + y_n) + 4(y_1 + y_3 + \dots) + 2(y_2 + y_4 + \dots)]$. Requires n to be even (a multiple of 2).
- **Simpson's 3/8 Rule:** Uses cubic polynomial segments over groups of 3 subintervals. Formula: $I = (3h/8)[(y_0 + y_n) + 3(y_1 + y_2 + y_4 + y_5 + \dots) + 2(y_3 + y_6 + \dots)]$. Requires n to be a multiple of 3.

Generally, Simpson's rules are more accurate than the Trapezoidal rule for the same number of points, because they fit curves (parabolas/cubics) rather than straight lines.

9. Tolerance & Step Size — Significance

- **Tolerance (Tol)** is the maximum acceptable error between successive approximations (or between the approximation and zero, in root-finding) before the algorithm stops iterating. It controls the trade-off between accuracy and computation time — a smaller tolerance means more iterations but a more precise answer. The worked examples consistently stop once $|x_{n+1} - x_n| < \text{Tol}$ (e.g., 10^{-4} , 10^{-6}).

Step Size (h) is the spacing between successive x -values used in interpolation, differentiation, or integration. Its significance:

- Smaller h generally reduces **truncation error** (the method becomes a better approximation of the true continuous function)
- But too small an h can increase **round-off error** due to floating-point subtraction of nearly equal numbers
- h also determines computational cost — smaller h means more function evaluations
- In the Richardson extrapolation example, halving h ($0.2 \rightarrow 0.1$) and combining both results gives a much more accurate estimate than either alone.

10. Limitations of Each Root-Finding Algorithm

- **Bisection Method:** Slow (linear convergence); requires an initial bracket with opposite signs; cannot find roots where f just touches zero without crossing (even-multiplicity roots); doesn't use any information about the function's shape.
- **False Position Method:** Also requires a bracket with opposite signs; can converge slowly if the function is strongly curved, because one endpoint may remain nearly fixed over many iterations.
- **Newton-Raphson Method:** Requires computing the derivative $f'(x)$, which may be difficult or costly; can diverge or fail if $f'(x) = 0$ near the guess; sensitive to a poor initial guess; may oscillate between two points or jump to an unrelated root.
- **Fixed Point Iteration:** Convergence depends entirely on choosing a rearrangement $x = g(x)$ with $|g'(x)| < 1$ — not every rearrangement works, and finding a suitable one isn't always straightforward; convergence can be slow (linear) even when it works.

11. Romberg Integration — Definition

Romberg Integration is a technique that improves the accuracy of numerical integration (typically starting from the Trapezoidal rule) by applying **Richardson extrapolation** repeatedly. It computes the Trapezoidal rule estimate at several step sizes ($h, h/2, h/4, \dots$) and combines these estimates in a systematic table to cancel out lower-order error terms, producing a much more accurate result with each additional level — conceptually the same idea as the Richardson extrapolation shown in the course material, but applied to integration rather than differentiation.